Questions

Questions::

==========
1. Application Overview
==========
- Tell me about your application

Our application is an internal auditing app used by companies to perform audit workflow in the proper manner.

Audit is basically maintaining internal information about an entity
For ex your finance department auditing will include fields like risks ,control ,documents

- Who uses it?
Senior/junior auditors ,directors ,managers

- Why do they use it?

To have an insight of what going on inside the company

==========
2. Architecture & Design
==========
- Architecture of the application

Our application follows a hybrid architecture and is currently in a transition phase
from a monolithic system to a microservices-based architecture.

At present, we have a single monolithic application that acts as the primary API Gateway.
It serves as the main entry point for client requests and handles request routing,
authentication, and some shared business logic.

Alongside the monolith, we have identified and segregated certain business functionalities
into independent microservices. These microservices are gradually being extracted from
the monolith and communicate with it through REST APIs and asynchronous messaging where required.

On the frontend, we use React with TypeScript for building the user interface, Redux for
state management, and Vite as the build tool to ensure fast development and optimized builds.

For infrastructure and supporting components:
- Redis is used for caching and performance optimization.
- Kafka is used for asynchronous event-driven communication.
- Docker is used for containerizing application components.
- Kubernetes is used for container orchestration and auto-scaling.
- AWS EC2 is used for hosting application services.
- AWS Lambda is used for specific event-driven or background tasks.
- AWS S3 is used for secure document and file storage.

- Splunk is used for centralized logging.
- Datadog is used for monitoring, metrics, and alerting.

Currently, the application uses a single shared database, which is typical during the initial phase of migrating from a monolith to microservices. This setup simplifies data consistency and reduces operational complexity while the migration is ongoing.

Overall, this architecture enables us to maintain system stability, support scalability, and modernize the application incrementally without impacting existing users.

- Why this architecture?

We chose this hybrid architecture because the application was originally built as a monolith and was already serving active users. Moving directly to a full microservices architecture would have been risky, time-consuming, and could have impacted production stability.

By keeping the monolith as the central entry point and gradually extracting selected modules into microservices, we can modernize the system incrementally without disrupting existing functionality.

This approach allows us to:
- Avoid a big-bang rewrite
- Reduce production risk
- Continue feature development during migration
- Scale only high-traffic or critical modules as microservices
- Validate microservices in production step by step

n6-
Using a single shared database during the transition simplifies data consistency and reduces operational complexity

Que on Archetecure::

1) which services converted to microservices and why?
-> We first extracted reporting, document management, notification, and background job modules into microservices. These modules were either resource-intensive, event-driven, or required independent scalability.
Core transactional audit workflows remained in the monolith to maintain data consistency and reduce migration risk during the transition phase**.**

2) how do you debug and which software do you use ?

a) **Synchronous (restTemplate)**
b) **Asynchronous(web client)**
c) **kafka**

- Advantages of the architecture

1. Gradual Migration
- Enables step-by-step transition from monolith to microservices
- Avoids risky big-bang rewrites

2. System Stability
- Existing monolith continues to serve users
- No disruption to critical business workflows

3. Independent Scalability
- High-traffic or critical modules can be scaled independently
- Kubernetes supports auto-scaling where required

4. Faster Time to Market
- New features can be developed alongside migration
- Teams are not blocked by architectural changes

5. Reduced Operational Risk
- Issues in new microservices do not impact the entire system
- Easier rollback during failures

6. Better Resource Utilization
- Microservices consume resources only when needed
- Redis and Kafka reduce unnecessary load on core services

7. Improved Observability
- Centralized logging with Splunk
- Real-time monitoring and alerts via Datadog

8. Cloud Readiness
- Docker and Kubernetes make the system cloud-native
- AWS services enable elasticity and scalability

9. Performance Optimization
- Caching with Redis reduces database load
- Event-driven processing via Kafka improves responsiveness

10. Future Scalability
- Architecture is ready for full microservices adoption
- Database separation and service isolation can be added incrementally

- Disadvantages of the architecture

1. Increased Complexity

- Managing both monolith and microservices increases architectural complexity
- Requires careful coordination between components

2. Single Database Limitation
- Shared database creates tight coupling between services
- Limits independent schema evolution
- Becomes a scalability bottleneck

3. Operational Overhead
- Requires DevOps effort for Docker, Kubernetes, monitoring, and logging
- More moving parts to manage and maintain

4. Network Latency
- Inter-service communication introduces network calls
- Slower compared to in-process monolith calls

5. Data Consistency Challenges
- Harder to manage transactions across services
- Distributed transaction handling is complex

6. Debugging Difficulty
- Issues can span multiple services
- Requires strong logging, tracing, and correlation IDs

7. Partial Microservices Benefits
- Cannot fully leverage microservices advantages due to shared database
- True service isolation is not achieved yet

8. Learning Curve
- Teams need expertise in cloud, containers, and distributed systems
- Higher onboarding time for new developers

9. Cost Overhead
- Kubernetes, monitoring tools, and cloud resources increase infrastructure cost

10. Dependency Management
- Monolith remains a critical dependency
- Failure in the monolith can still impact the entire system

==========
3. API Gateway & Traffic Management
==========

Have you used an API Gateway?

Yes, we use a dedicated API Gateway service in our microservices architecture.

Instead of the monolith acting as the entry point, we implemented a separate API Gateway layer that serves as the single entry point for all client requests.

The API Gateway is responsible for:

- Routing requests to appropriate backend microservices
- Authentication and JWT validation
- Authorization enforcement
- Centralized logging and monitoring
- Request validation
- Rate limiting and throttling
- Handling cross-cutting concerns

This separation improves:

- Scalability
- Security
- Maintainability
- Clear separation of concerns

It also allows backend services to remain lightweight and focused only on business logic.

✅ How does the API Gateway manage load?

The API Gateway acts as a centralized traffic controller.

1️⃣ Load Balancing

- It routes incoming traffic to multiple instances of backend services.
- In Kubernetes deployment, it works with Kubernetes Services and kube-proxy for load balancing.
- Only healthy pods receive traffic (health checks enabled).

2️⃣ Rate Limiting

Rate limiting restricts how many requests a client can send within a specific time window.

Purpose:

- Prevent abuse
- Avoid DDoS-style traffic spikes
- Ensure fair usage among clients

Example:

- 100 requests per minute per user

If limit exceeds → 429 Too Many Requests

3️⃣ Throttling

Throttling controls request processing speed when traffic exceeds thresholds.

Instead of rejecting immediately:

- It slows down request processing
- Protects backend services
- Maintains system stability

Throttling is especially useful during sudden traffic spikes.

4️⃣ Auto-Scaling

Auto-scaling happens at the infrastructure level (Kubernetes).

- Horizontal Pod Autoscaler increases service instances
- Based on CPU, memory, or request count
- Scales down during low traffic to optimize cost

So load management is a combination of:

- API Gateway traffic control
- Kubernetes load balancing

Throttling and rate limiting are handled at the API Gateway level.

How is load balancing done in your project?
Architecture Short Note – Load Balancing, Service Registry & Routing (Kubernetes Deployment)

1️⃣ Who does Load Balancing?

In Kubernetes, load balancing is handled at multiple layers:

• Kubernetes Service (ClusterIP / NodePort / LoadBalancer)
- Distributes traffic across healthy pods.
- Uses kube-proxy (iptables/IPVS) internally.
- Performs round-robin load balancing by default.

• Ingress Controller (if used)
- Handles external HTTP/HTTPS traffic.
- Routes traffic from outside cluster to internal services.

• API Gateway (Application Level)
- Routes requests to correct backend services.
- Applies rate limiting, throttling, authentication.
- Relies on Kubernetes Service for actual pod-level balancing.

So:

External LB → Ingress → API Gateway → Kubernetes Service → Pods

---

2️⃣ Do we use Service Registry?

In Kubernetes, a separate service registry like Eureka is usually NOT required.

Reason:
- **Kubernetes has built-in service discovery.**
- **Each Service gets a DNS name.**
- **Pods communicate using:**

  http://service-name:port

Kube-DNS / CoreDNS resolves service names to cluster IPs automatically.

So Kubernetes itself acts as:
- **Service Registry**
- **Service Discovery mechanism**

3️⃣ How is Routing Done?

Routing happens in layers:

A) External Routing
Client → Load Balancer → Ingress Controller

B) Gateway Routing
API Gateway inspects:
- Path (/audit/)**
- HTTP method
- Headers
- JWT claims

Then forwards to appropriate internal service.

C) Internal Routing
API Gateway calls:
http://audit-service:8080
http://auth-service:8080

Kubernetes Service forwards request to one of the healthy pods.

Final Flow:

Client
↓
Cloud Load Balancer
↓
Ingress Controller
↓
API Gateway
↓
Kubernetes Service (DNS-based discovery)
↓
Pod Instance (Load Balanced)

Key Points for Interview:

- **Kubernetes handles service discovery.**
- **kube-proxy handles pod-level load balancing.**
- **Ingress handles external routing.**
- **API Gateway handles application-level routing and traffic control.**
- **No separate Eureka needed in Kubernetes-native setup.**

==========
4. Database Design & Load Handling
==========

- How many databases do you have?

Currently, we use a single shared database across the application.

This is intentional because the system is in a transition phase from a monolithic architecture to microservices. Using a single database simplifies data consistency, reduces migration complexity, and ensures stability while services are being gradually extracted.

As part of the future roadmap, the plan is to move toward a database-per-service model once the microservices are fully isolated.

- How does your database handle load?

The database handles load through a combination of design optimizations and supporting infrastructure.

We use proper indexing on frequently queried columns to speed up read operations. Connection pooling is enabled to efficiently manage database connections and prevent connection exhaustion under high traffic.

To reduce direct database load, Redis caching is used for frequently accessed and read-heavy data. Pagination is applied for large result sets to avoid heavy queries.

Additionally, background processing and asynchronous workflows via Kafka help offload non-critical operations from synchronous database access, improving overall performance and scalability.

Connection Pooling

Connection pooling works by maintaining a pool of pre-created database connections that can be reused by multiple requests instead of creating a new connection each time.

When the application starts, the connection pool (for example, HikariCP in Spring Boot) creates a fixed number of database connections and keeps them ready.

When a request needs to access the database:
- It borrows an available connection from the pool
- Executes the query
- Returns the connection back to the pool instead of closing it

If all connections are in use, new requests wait until a connection becomes available, rather than creating unlimited connections and overwhelming the database.

This approach:
- Reduces connection creation overhead
- Improves performance under high traffic
- Prevents database connection exhaustion
- Ensures controlled and efficient use of database resources

==========
5. Caching Strategy
==========
- Is caching used in your project?
Yes, caching is used in our project.
We use Redis as a centralized in-memory cache to improve performance and reduce load on the database. Caching is mainly applied to frequently accessed, read-heavy data that does not change frequently.

- What is stored in the cache?
- When is data stored in cache?
- When and how is cache cleared?

==========
6. Logging & Monitoring
==========
- How is logging implemented?
- Which logging/monitoring tools are used?
- How is the logging tool integrated?
- How does application data reach the logging system?

==========
7. Authentication & Authorization
==========
- How is authentication managed?
- How is authorization managed?

==========
8. Inter-Service Communication
==========
- How do services communicate with each other?
- Synchronous vs asynchronous communication
- where is multithreading applied in your application ?

==========
9. Messaging / Event Streaming
==========
- Where is Kafka used in your project?
- Why is Kafka used?

==========
10. Background & Scheduled Jobs
==========
- Are there background jobs in your project?
- What kind of tasks run in the background?

==========
11. Bugs & Issue Handling
==========
- Bugs faced in the project
- How were those bugs identified?
- How were they resolved?

==========
12. Recent Contributions
==========
- Your recent contributions to the project

==========
13. Roles & Responsibilities
==========
- Your role in the team
- Your day-to-day responsibilities

==========
14. Challenges & Problem Solving
==========
- Difficulties faced in the project
- How did you tackle them?

==========
15. AOP (Aspect-Oriented Programming)
==========
- Where is AOP used in your project?
- Why was AOP used?

=====
Cloud
=====
- how do you manage load balancing ?

========
Others
========

1) **HTTP vs HTTPS**

Functional Info

We have an **internal audit application** that supports the **end-to-end audit lifecycle**.

Key modules:

- **Dashboard**
  Central overview screen showing audit status, pending tasks, KPIs, notifications, and recent activities. Helps users quickly understand overall system health and workload.
- **Assessment Area**
  Module where auditors create and manage audit assessments, risks, controls, findings, and observations. It acts as the core workspace for audit execution.
- **Project Management**
  Used to plan audits, assign team members, manage deadlines, milestones, and track project progress. Helps coordinate audit activities across teams.
- **Time Tracking**
  Allows auditors to log hours spent on audit tasks and activities. Used for productivity analysis, billing, utilization, and compliance reporting.
- **Audit Report Creation**
  Generates final audit reports containing findings, risk levels, recommendations, and approvals. Supports exporting reports in formats like PDF or Excel.
- **Recycle Bin**
  Stores deleted audit records temporarily for recovery before permanent deletion. Helps prevent accidental data loss.
- **Audit History**
  Maintains complete audit trail of changes, actions, approvals, and updates performed in the system. Important for compliance, traceability, and accountability.
-

Core audit objects:

- Risks and controls
- Strategic risks
- Objectives
- Workpapers

Major functionalities:

- Configurable **workflows**
- **Dynamic role- and permission-based access control**
- **Dynamic rule engine** for business logic
- **Workpaper upload and download**
- Customizable **naming and taxonomy**
- **SMTP-based email notifications**
- Automated **audit report generation**

Competitors::

AuditBoard
Workiva

Business user/admin flow

Admin Flow (Context Definition)

In the audit application, the **Admin** has full system-level access and configuration control.

Admin responsibilities include:

- Managing **taxonomy settings** across the application
- Defining **roles and permissions**
    - Creating new roles from existing system roles
    - Assigning permissions to roles
- **User management**
    - Creating users
    - Assigning roles to users
- Customizing the system
    - Renaming audit objects (risk, control, objective, workpaper, etc.)
    - Modifying taxonomy structures
- Configuring **SMTP settings** for email notifications

In addition to administrative capabilities, the **Admin can perform all operations available to a normal user**, including working on audits, assessments, workpapers, workflows, and reports.

User Flow (Context Definition)

A **User** has functional access based on the roles and permissions assigned to them.

User capabilities include:

- Accessing the **Assignment area**
    - Creating new assignments
    - Adding audit objects within an assignment
- Working with **Projects**
    - Linking a project to a specific audit area
    - Performing audits within that project
    - Adding predefined audit objects (risks, controls, objectives, workpapers, etc.)
- Managing audit objects
    - Creating, editing, and deleting objects
- Generating **audit reports** from assignment or project data
- Participating in **assessment workflows**
    - Being assigned to specific assessment phases
    - Receiving notifications based on assigned phases and tasks

User limitations:

- No access to **administrative areas** such as:
    - System settings
    - Statistics
    - Configuration modules

Application flow

TeamMate+ Audit – Overview

TeamMate+ Audit is a software application used by internal audit teams to **plan, execute, track, and report audits** efficiently.
It is widely used by organizations—especially in regulated industries like **banking**—to manage **risk, compliance, and governance** activities.

Who Uses TeamMate+ Audit?

1. Internal Audit Teams

- Conduct audits across departments
- Evaluate internal controls and processes
- Identify gaps, risks, and improvement areas

2. Risk Management Professionals

- Identify and assess operational, financial, and compliance risks
- Ensure risks are monitored and mitigated

3. Compliance Officers

- Ensure adherence to regulatory requirements and internal policies
- Track compliance issues and corrective actions

4. Audit Managers & Supervisors

- Plan audits and allocate resources
- Monitor audit progress and quality
- Review findings and final reports

Why Do Organizations Use TeamMate+ Audit?

Streamlined Audit Processes

- End-to-end audit lifecycle management
- Reduces manual work and spreadsheets

Enhanced Collaboration

- Centralized platform for auditors and stakeholders
- Clear visibility into tasks, findings, and status

Risk-Based Auditing

- Focuses audits on high-risk areas
- Improves early risk detection

Compliance Assurance

- Ensures audits align with regulatory standards
- Maintains audit trails and documentation

Banking Sector Example (Simple Explanation)

Q1. Who Uses TeamMate+ Audit in a Bank?

- **Internal Audit Team**
  Checks whether banking operations are safe, efficient, and compliant.
- **Risk Management Team**
  Identifies risks like fraud, credit risk, system failures, and operational issues.
- **Compliance Officers**
  Ensure adherence to RBI / central bank regulations and internal policies.

Q2. Why Do Banks Use TeamMate+ Audit?

Banks handle:

- Huge volumes of transactions
- Sensitive customer data
- Strict regulatory requirements

They must ensure:

- **Operational safety** – no fraud or transaction errors
- **Regulatory compliance** – RBI / central bank rules
- **Risk minimization** – early detection of issues
- **Accurate audit reporting** – for regulators and senior management

TeamMate+ Audit helps **automate, standardize, and control** these activities.

Q3. How Does a Bank Use TeamMate+ Audit?

1. Audit Planning

- Decide audit areas (loans, deposits, treasury, IT systems)
- Schedule audits and assign auditors
- Prioritize audits based on risk levels

2. Risk Assessment

- Identify high-risk areas:
    - Credit risk
    - Fraud risk
    - Operational risk
- Allocate more audit effort to critical areas

3. Audit Execution

- Auditors review transactions and controls
- Evidence and findings stored centrally
- Eliminates paperwork and scattered files

4. Reporting

- Automatically generates audit reports
- Clearly highlights:
    - Issues
    - Risks
    - Recommendations

5. Follow-Up (Cron-Based Tracking)

- Tracks whether audit issues are resolved
- Sends reminders until actions are closed
- Ensures accountability

Benefits to the Bank

- **Time Savings** – Less manual tracking and documentation
- **Reduced Risk** – Systematic and consistent audits
- **Improved Compliance** – Easier regulatory adherence
- **Actionable Insights** – Identifies recurring issues and trends
- **Transparency** – Clear visibility for management and regulators

---

Final Summary

✅ **In short:**
In the banking sector, **TeamMate+ Audit** is used to **manage audits, control risks, ensure regulatory compliance, and improve audit efficiency**, helping banks remain **safe, compliant, and trustworthy**.

Technical Architecture

Tech stack used

Frontend:
- React 18, TypeScript, Vite
- Redux Toolkit

Backend:
- Java 17
- Spring Boot 3.x
- Spring Security, JWT, OAuth2
- RESTful APIs, Swagger (OpenAPI)
- JPA / Hibernate

Database:
- MySQL

Messaging & Cache:
- Kafka
- Redis

DevOps & Cloud:
- Git, GitHub
- Jenkins (CI/CD pipelines)
- Docker, Kubernetes
- AWS (EC2, S3, Lambda)

Testing & Quality:
- JUnit, Mockito
- Spring Boot Test
- SonarQube

Monitoring & Logging:
- Splunk
- Datadog

Tools:
- MobaXterm
- PuTTY

Design Pattern